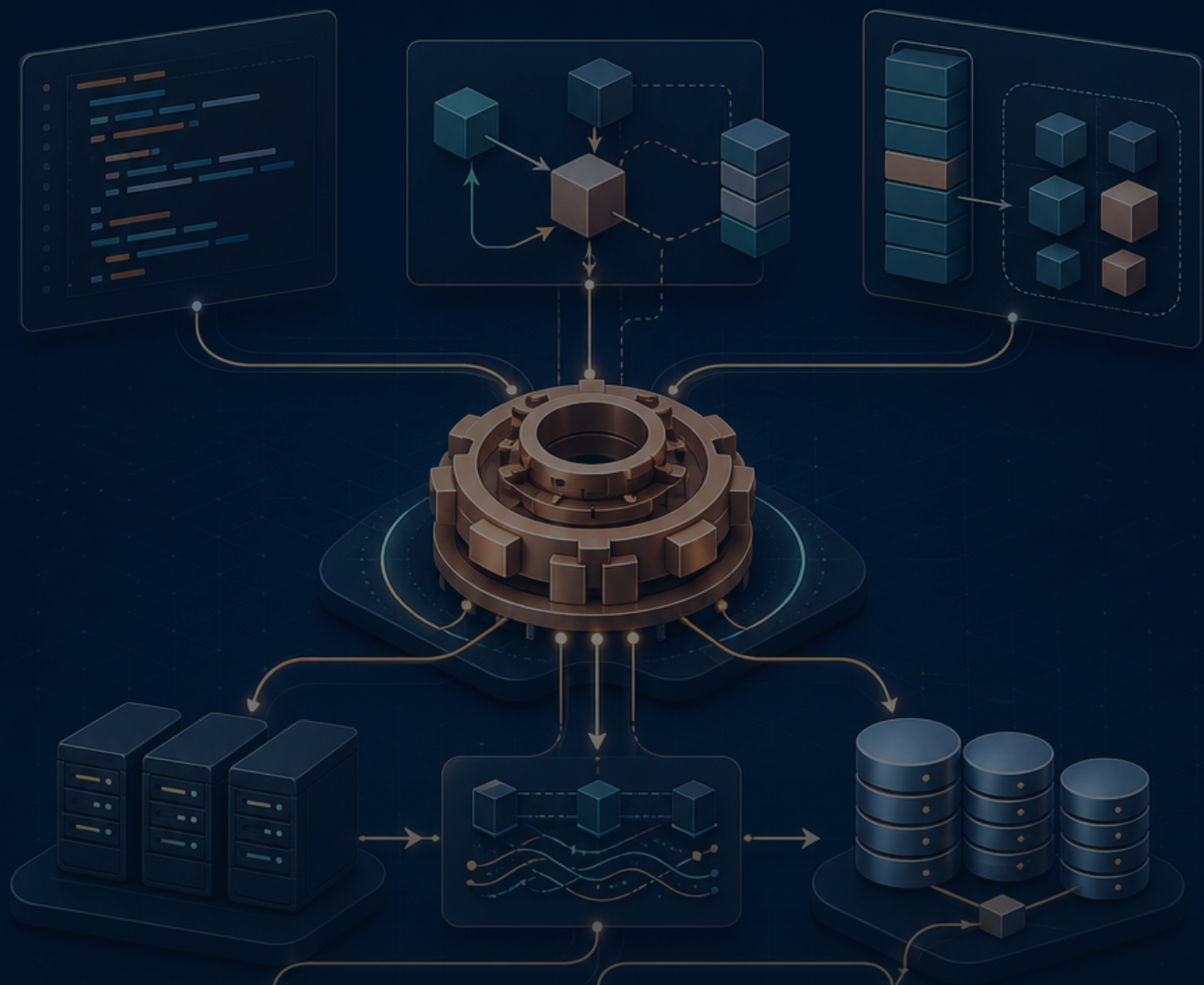




VOLUME 03 / RUST BACKEND MASTERY

Rust中核 — 所有権・借用・ライフタイム・スマートポイント

メモリ図とコンパイラエラーを往復し、なぜ安全なのかをバックエンドの共有状態までつなげる

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

Moveを規則として暗記せず、誰が解放責任を持つか、参照がいつまで有効か、スレッド間で何を共有できるかを図解します。

到達目標

- ✓ Move・Copy・Cloneの費用と意図を説明できる
- ✓ &Tと&mut Tの排他規則をコードへ適用できる
- ✓ ライフタイムを参照関係として読める
- ✓ Box・Rc・Arcと内部可変性を用途別に選べる

学習地図

01 スタック・ヒープ・スコープ・Drop

固定サイズの局所値は主にスタックへ、実行時に大きさが変わるStringやVecの本体データはヒープへ置かれます。所有する変数がスコープを抜けるとDropが一度だけ走ります。

02 Move・Copy・Clone

Stringの代入はポインタ情報の複製後に元を無効化するMoveで、二重解放を防ぎます。整数などCopy型はビット単位複製され、Cloneは費用を明示した複製です。

03 共有参照&Tと可変参照&mut T

参照は値を所有せず一時的に借ります。同時に複数の共有参照、または一つの可変参照のどちらかだけを許す規則が、データ競合と無効参照をコンパイル時に防ぎます。

04 スライスと借用APIの設計

&strと&[T]は所有データの一部分を借りるfat pointerです。開始位置に加えて長さを持ち、元データより長く生きられません。APIが所有を必要としないならスライスを受け取ります。

05 ライフタイム注釈と省略規則

ライフタイムは参照が有効な期間の関係を表し、値を長生きさせる記法ではありません。多くは省略規則で推論され、複数参照から一つを返すときに関係を注釈します。

06 参照を持つstruct・static・所有へ切替

参照フィールドを持つstructには、その参照がstructより長く有効である関係が必要です。'staticはプログラム全体に有効な参照であり、何にでも付ける解決策ではありません。

07 NLL・再借用・借用範囲を短くする

Non-Lexical Lifetimesにより借用はブロック末尾ではなく最後の使用箇所です。それでも長い&mut借用は他の操作を妨げるため、検索と更新を短い式へ閉じ込めます。

08 Box<T>・再帰型・trait object

Box<T>は値をヒープへ置き、ポインタ自体は固定サイズになります。再帰型のサイズを有限にし、大きな値の所有権移動を軽くし、dyn Traitの実体を保持できます。

09 Rc<T>・RefCell<T>・内部可変性

Rc<T>は単一スレッドで所有者を複数にし、RefCell<T>は借用規則を実行時に検査します。コンパイル時保証を弱めるため、UI木やテスト用Fakeなど限定した場所で使います。

10 Arc<T>・Mutex<T>・Send・Sync

Arc<T>はスレッド安全な参照カウント、Mutex<T>は共有可変状態への排他アクセスを提供します。Sendは所有権を別スレッドへ移せること、Syncは&Tを複数スレッドで共有できることを表します。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

スタックフレーム

STEP 2

ヒープ割り当て

STEP 3

スコープ

STEP 4

DropとRAII

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

スタック・ヒープ・スコープ・Drop

UNIT 01・CONCEPT

スタック・ヒープ・スコープ・Drop

固定サイズの局所値は主にスタックへ、実行時に大きさが変わるStringやVecの本体データはヒープへ置かれます。所有する変数がスコープを抜けるとDropが一度だけ走ります。

この単元の到達点

- スタックフレーム
- ヒープ割り当て
- スコープ
- DropとRAII

なぜバックエンドで必要か

DB接続、ファイル、ロックはRAIIでスコープ終了時に解放されます。解放処理を手動の全return経路へ書く必要がありません。

入力・所有する側

スタックフレーム	owner
ヒープ割り当て	owner

処理・借りる側

スコープ	borrow
DropとRAII	borrow

先に答え

所有者のスコープ終了で資源を一度だけ解放する仕組みがRAIIです。

確認する問い

スタック・ヒープ・スコープ・Dropを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 01・MENTAL MODEL

スタック・ヒープ・スコープ・Dropを図でつかむ

入力・所有する側

スタックフレーム	owner
ヒープ割り当て	owner

処理・借りる側

スコープ	borrow
DropとRAII	borrow

固定サイズの局所値は主にスタックへ、実行時に大きさが変わるStringやVecの本体データはヒープへ置かれます。所有する変数がスコープを抜けるとDropが一度だけ走ります。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	スタックフレーム	スタックフレーム — 入力と前提を確認し、境界を曖昧にしない。
2	ヒープ割り当て	ヒープ割り当て — 値がどの順で変換されるかを追跡する。
3	スコープ	スコープ — 失敗を再現し、型またはログから原因を特定する。
4	DropとRAII	DropとRAII — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

スタック・ヒープ・スコープ・Dropとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

スタック・ヒープ・スコープ・Dropだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
struct Connection(&'static str);
impl Drop for Connection {
    fn drop(&mut self) { println!("close {}", self.0); }
}
fn main() {
    println!("before");
    { let _db = Connection("database"); println!("inside"); }
    println!("after");
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>struct Connection(&'static str);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>impl Drop for Connection {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn drop(&mut self) { println!("close {}", self.0); }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力がスタックフレームの条件を満たすか確認する。
- ✓ ヒープ割り当てを実行し、途中の型と状態を追う。
- ✓ スコープで失敗を成功経路から分離する。
- ✓ DropとRAIIにより、出力と副作用を検証する。

型を声に出す スタック・ヒープ・スコープ・Dropに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
before
inside
close database
after
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
struct Guard(&'static str);
impl Drop for Guard { fn drop(&mut self) { println!("drop {}", self.0); } }
fn main() {
    let _outer = Guard("outer");
    { let _inner = Guard("inner"); }
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

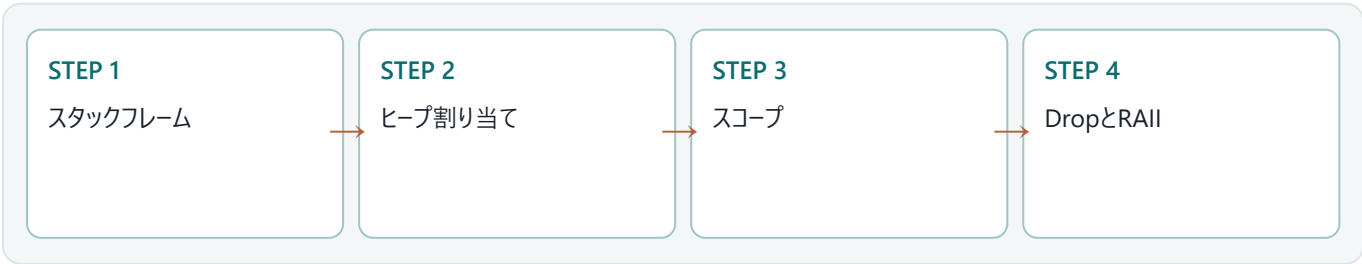
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

DB接続、ファイル、ロックはRAIIでスコープ終了時に解放されます。解放処理を手動の全return経路へ書く必要がありません。



リクエスト側

- スタックフレームを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- ヒープ割り当てを小さな責務へ分割
- スコープをResultとログで表現
- DropとRAIIをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	スタックフレームに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違ふ	ヒープ割り当ての入力または実行順序が想定と違ふ	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	スコープまたはDropとRAIIの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

スタック・ヒープ・スコープ・Dropを再現して切り分ける

スタック・ヒープ・スコープ・Dropの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: スタックフレーム
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: ヒープ割り当て
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: スコープ
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: DropとRAII

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	スタックフレーム	スタックフレーム — 入力と前提を確認し、境界を曖昧にしない。
2	ヒープ割り当て	ヒープ割り当て — 値がどの順で変換されるかを追跡する。
3	スコープ	スコープ — 失敗を再現し、型またはログから原因を特定する。
4	DropとRAII	DropとRAII — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 所有者のスコープ終了で資源を一度だけ解放する仕組みがRAIIです。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 · PRACTICE

手を動かす演習

一時ファイルを表す型へDropを実装し、ネストしたスコープで破棄順序を観察してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
struct Guard(&'static str);
impl Drop for Guard { fn drop(&mut self) { println!("drop {}", self.0); } }
fn main() {
    let _outer = Guard("outer");
    { let _inner = Guard("inner"); }
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 所有者のスコープ終了で資源を一度だけ解放する仕組みがRAIIです。
- ✓ スタック・ヒープ・スコープ・Dropとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

Move・Copy・Clone

UNIT 02 ・ CONCEPT

Move・Copy・Clone

Stringの代入はポインタ情報の複製後に元を無効化するMoveで、二重解放を防ぎます。整数などCopy型はビット単位複製され、Cloneは費用を明示した複製です。

この単元の到達点

- Move後の無効化
- Copy trait
- Cloneの費用
- 所有権を渡すAPI

なぜバックエンドで必要か

DTOをユースケースへMoveすると、その後の誤使用を防げます。大きなVecを無意識にcloneするとリクエストごとのCPU・メモリ負荷になります。

入力・所有する側

Move後の無効化	owner
Copy trait	owner

処理・借りる側

Cloneの費用	borrow
所有権を渡すAPI	borrow

先に答え

所有権を移すのは責務移譲、借りるのは一時利用、cloneは費用を払う複製です。

確認する問い

Move・Copy・Cloneを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 02 ・ MENTAL MODEL

Move・Copy・Cloneを図でつかむ

入力・所有する側

Move後の無効化	owner
Copy trait	owner

処理・借りる側

Cloneの費用	borrow
所有権を渡すAPI	borrow

Stringの代入はポインタ情報の複製後に元を無効化するMoveで、二重解放を防ぎます。整数などCopy型はビット単位複製され、Cloneは費用を明示した複製です。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Move後の無効化	Move後の無効化 — 入力と前提を確認し、境界を曖昧にしない。
2	Copy trait	Copy trait — 値がどの順で変換されるかを追跡する。
3	Cloneの費用	Cloneの費用 — 失敗を再現し、型またはログから原因を特定する。
4	所有権を渡すAPI	所有権を渡すAPI — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Move・Copy・Cloneとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 · MINIMUM CODE

まず動く最小コード

Move・Copy・Cloneだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn consume(value: String) { println!("consumed={value}"); }
fn main() {
    let title = String::from("ownership");
    let backup = title.clone();
    consume(title);
    println!("backup={backup}");
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn consume(value: String) { println! ("consumed={value}"); }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let title = String::from("ownership");</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let backup = title.clone();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>consume(title);</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 Move 後の無効化の条件を満たすか確認する。
- ✓ Copy trait を実行し、途中の型と状態を追う。
- ✓ Clone の費用で失敗を成功経路から分離する。
- ✓ 所有権を渡す API により、出力と副作用を検証する。

型を声に出す Move・Copy・Clone に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
consumed=ownership
backup=ownership
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn with_len_owned(value: String) -> (String, usize) {
    let len = value.len();
    (value, len)
}
fn len_borrowed(value: &str) -> usize { value.len() }
fn main() {
    let s = String::from("rust");
    assert_eq!(len_borrowed(&s), 4);
    let (s, len) = with_len_owned(s);
    assert_eq!((s.as_str(), len), ("rust", 4));
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

DTOをユースケースへMoveすると、その後の誤使用を防げます。大きなVecを無意識にcloneするとリクエストごとのCPU・メモリ負荷になります。



リクエスト側

- Move後の無効化を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- Copy traitを小さな責務へ分割
- Cloneの費用をResultとログで表現
- 所有権を渡すAPIをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Move後の無効化に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	Copy traitの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	Cloneの費用または所有権を渡すAPIの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error[E0382]: borrow of moved value: `title`
help: consider cloning the value if the performance cost is acceptable
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

Move・Copy・Cloneを再現して切り分ける

Move・Copy・Cloneの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Move後の無効化
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: Copy trait
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Cloneの費用
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 所有権を渡すAPI

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Move後の無効化	Move後の無効化 — 入力と前提を確認し、境界を曖昧にしない。
2	Copy trait	Copy trait — 値がどの順で変換されるかを追跡する。
3	Cloneの費用	Cloneの費用 — 失敗を再現し、型またはログから原因を特定する。
4	所有権を渡すAPI	所有権を渡すAPI — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 所有権を移すのは責務移譲、借りるのは一時利用、cloneは費用を払う複製です。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

Stringを受け取って長さも返す関数を、cloneなしでタプル返却と借用の二通りで作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn with_len_owned(value: String) -> (String, usize) {
    let len = value.len();
    (value, len)
}
fn len_borrowed(value: &str) -> usize { value.len() }
fn main() {
    let s = String::from("rust");
    assert_eq!(len_borrowed(&s), 4);
    let (s, len) = with_len_owned(s);
    assert_eq!((s.as_str(), len), ("rust", 4));
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 所有権を移すのは責務移譲、借りるのは一時利用、cloneは費用を払う複製です。
- ✓ Move・Copy・Cloneとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

共有参照& Tと可変参照&mut T

UNIT 03 · CONCEPT

共有参照& Tと可変参照&mut T

参照は値を所有せず一時的に借ります。同時に複数の共有参照、または一つの可変参照のどちらかだけを許す規則が、データ競合と無効参照をコンパイル時に防ぎます。

この単元の到達点

- 共有借用
- 排他的な可変借用
- 借用の範囲
- データ競合の防止

なぜバックエンドで必要か

複数ハンドラが共有する状態は不変参照で読み、更新はDBトランザクションやMutexなど排他境界へ集めます。

入力・所有する側

共有借用	owner
排他的な可変借用	owner

処理・借りる側

借用の範囲	borrow
データ競合の防止	borrow

先に答え

読む借用は共有でき、書く借用は排他的です。借用範囲を短く保ちます。

確認する問い

共有参照& Tと可変参照&mut Tを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 03 · MENTAL MODEL

共有参照& Tと可変参照&mut Tを図でつかむ

入力・所有する側

共有借用	owner
排他的な可変借用	owner

処理・借りる側

借用の範囲	borrow
データ競合の防止	borrow

参照は値を所有せず一時的に借ります。同時に複数の共有参照、または一つの可変参照のどちらかだけを許す規則が、データ競合と無効参照をコンパイル時に防ぎます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	共有借用	共有借用 — 入力と前提を確認し、境界を曖昧にしない。
2	排他的な可変借用	排他的な可変借用 — 値がどの順で変換されるかを追跡する。
3	借用の範囲	借用の範囲 — 失敗を再現し、型またはログから原因を特定する。
4	データ競合の防止	データ競合の防止 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

共有参照& Tと可変参照&mut Tとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03 · MINIMUM CODE

まず動く最小コード

共有参照& Tと可変参照&mut Tだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn normalize(value: &str) -> String { value.trim().to_lowercase() }
fn mark_done(done: &mut bool) { *done = true; }
fn main() {
    let title = String::from(" Rust ");
    let normalized = normalize(&title);
    let mut done = false;
    mark_done(&mut done);
    println!("{normalized} done={done} original={title}");
}
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn normalize(value: &str) -> String { value.trim().to_lowercase() }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn mark_done(done: &mut bool) { *done = true; }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let title = String::from(" Rust ");</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let normalized = normalize(&title);</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力共有借用の条件を満たすか確認する。
- ✓ 排他的な可変借用を実行し、途中の型と状態を追う。
- ✓ 借用の範囲で失敗を成功経路から分離する。
- ✓ データ競合の防止により、出力と副作用を検証する。

型を声に出す 共有参照& Tと可変参照&mut Tに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

rust done=true original= Rust

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Debug)] struct Note { id: u64, done: bool }
fn find(notes: &[Note], id: u64) -> Option<&Note> { notes.iter().find(|n| n.id == id) }
fn toggle(notes: &mut [Note], id: u64) -> bool {
    if let Some(note) = notes.iter_mut().find(|n| n.id == id) { note.done = !note.done; true } else { false }
}
fn main() { let mut notes = vec![Note { id: 1, done: false }]; assert!(toggle(&mut notes, 1)); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 ・ BACKEND APPLICATION

バックエンドのどこで使うか

複数ハンドラが共有する状態は不変参照で読み、更新はDBトランザクションやMutexなど排他境界へ集めます。



リクエスト側

- 共有借用を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- 排他的な可変借用を小さな責務へ分割
- 借用の範囲をResultとログで表現
- データ競合の防止をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	共有借用に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	排他的な可変借用の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	借用の範囲またはデータ競合の防止の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

共有参照& Tと可変参照&mut Tを再現して切り分ける

共有参照& Tと可変参照&mut Tの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 共有借用
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 排他的な可変借用
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 借用の範囲
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: データ競合の防止

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	共有借用	共有借用 — 入力と前提を確認し、境界を曖昧にしない。
2	排他的な可変借用	排他的な可変借用 — 値がどの順で変換されるかを追跡する。
3	借用の範囲	借用の範囲 — 失敗を再現し、型またはログから原因を特定する。
4	データ競合の防止	データ競合の防止 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 読む借用は共有でき、書く借用は排他的です。借用範囲を短く保ちます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

`Vec<Note>`を`&[Note]`で検索し、`&mut [Note]`で一件だけtoggleする二つの関数を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Debug)] struct Note { id: u64, done: bool }
fn find(notes: &[Note], id: u64) -> Option<&Note> { notes.iter().find(|n| n.id == id) }
fn toggle(notes: &mut [Note], id: u64) -> bool {
    if let Some(note) = notes.iter_mut().find(|n| n.id == id) { note.done = !note.done; true } else { false }
}
fn main() { let mut notes = vec![Note { id: 1, done: false }]; assert!(toggle(&mut notes, 1)); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 読む借用は共有でき、書く借用は排他的です。借用範囲を短く保ちます。
- ✓ 共有参照& Tと可変参照&mut Tとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

スライスと借用APIの設計

UNIT 04 · CONCEPT

スライスと借用APIの設計

`&str`と`&[T]`は所有データの一部分を借りるfat pointerです。開始位置に加えて長さを持ち、元データより長く生きられません。APIが所有を必要としないならスライスを受け取ります。

この単元の到達点

- 文字列スライス
- 配列スライス
- fat pointer
- 汎用的な借用引数

なぜバックエンドで必要か

フォームのStringを検証関数へ`&str`で渡し、DB行のVecを集計関数へ`&[T]`で渡すと、不要な所有権移動とcloneを避けられます。

入力・所有する側

文字列スライス	owner
配列スライス	owner

処理・借りる側

fat pointer	borrow
汎用的な借用引数	borrow

先に答え

所有を要求しない関数は`&str`や`&[T]`を受け、呼び出し側の選択肢を広げます。

確認する問い

スライスと借用APIの設計を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 04 · MENTAL MODEL

スライスと借用APIの設計を図でつかむ

入力・所有する側

文字列スライス	owner
配列スライス	owner

処理・借りる側

fat pointer	borrow
汎用的な借用引数	borrow

`&str`と`&[T]`は所有データの一部分を借りるfat pointerです。開始位置に加えて長さを持ち、元データより長く生きられません。APIが所有を必要としないならスライスを受け取ります。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	文字列スライス	文字列スライス — 入力と前提を確認し、境界を曖昧にしない。
2	配列スライス	配列スライス — 値がどの順で変換されるかを追跡する。
3	fat pointer	fat pointer — 失敗を再現し、型またはログから原因を特定する。
4	汎用的な借用引数	汎用的な借用引数 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

スライスと借用APIの設計とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE

まず動く最小コード

スライスと借用APIの設計だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn first_word(value: &str) -> &str {
    value.split_whitespace().next().unwrap_or("")
}
fn active_count(flags: &[bool]) -> usize {
    flags.iter().filter(|&&flag| *flag).count()
}
fn main() {
    let text = String::from("rust backend");
    println!("{}", first_word(&text), active_count(&[true, false, true]));
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn first_word(value: &str) -> &str {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>value.split_whitespace().next().unwrap_or("")</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn active_count(flags: &[bool]) -> usize {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>flags.iter().filter(&&flag flag).count()</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力文字列スライスの条件を満たすか確認する。
- ✓ 配列スライスを実行し、途中の型と状態を追う。
- ✓ fat pointerで失敗を成功経路から分離する。
- ✓ 汎用的な借用引数により、出力と副作用を検証する。

型を声に出す スライスと借用APIの設計に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

rust 2

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力値が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
struct Note { done: bool }
fn pending(notes: &[Note]) -> Vec<&Note> {
    notes.iter().filter(|n| !n.done).collect()
}
fn main() {
    let notes = vec![Note { done: false }, Note { done: true }];
    assert_eq!(pending(&notes).len(), 1);
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

フォームのStringを検証関数へ&strで渡し、DB行のVecを集計関数へ&[T]で渡すと、不要な所有権移動とcloneを避けられます。



リクエスト側

- 文字列スライスを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- 配列スライスを小さな責務へ分割
- fat pointerをResultとログで表現
- 汎用的な借用引数をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	文字列スライスに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	配列スライスの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	fat pointerまたは汎用的な借用引数の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

スライスと借用APIの設計を再現して切り分ける

スライスと借用APIの設計の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 文字列スライス
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 配列スライス
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: fat pointer
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 汎用的な借用引数

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	文字列スライス	文字列スライス — 入力と前提を確認し、境界を曖昧にしない。
2	配列スライス	配列スライス — 値がどの順で変換されるかを追跡する。
3	fat pointer	fat pointer — 失敗を再現し、型またはログから原因を特定する。
4	汎用的な借用引数	汎用的な借用引数 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 所有を要求しない関数は&strや&[T]を受け、呼び出し側の選択肢を広げます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

&[Note]を受け、未完了ノートだけをVec<&Note>として返す関数を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
struct Note { done: bool }
fn pending(notes: &[amp;Note]) -> Vec<&Note> {
    notes.iter().filter(|n| !n.done).collect()
}
fn main() {
    let notes = vec![Note { done: false }, Note { done: true }];
    assert_eq!(pending(&notes).len(), 1);
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 所有を要求しない関数は&strや&[T]を受け、呼び出し側の選択肢を広げます。
- ✓ スライスと借用APIの設計とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

ライフタイム注釈と省略規則

UNIT 05 · CONCEPT

ライフタイム注釈と省略規則

ライフタイムは参照が有効な期間の関係を表し、値を長生きさせる記法ではありません。多くは省略規則で推論され、複数参照から一つを返すときに関係を注釈します。

この単元の到達点

- 参照の有効期間
- dangling防止
- 省略規則
- 入力と出力の関係

なぜバックエンドで必要か

リクエストから借りた&strをアプリ全体状態へ保存できない理由を理解し、必要なら所有するStringへ変換します。

入力・所有する側

参照の有効期間	owner
dangling防止	owner

処理・借りる側

省略規則	borrow
入力と出力の関係	borrow

先に答え

ライフタイム注釈は参照同士の有効期間の関係を説明します。

確認する問い

ライフタイム注釈と省略規則を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 05 · MENTAL MODEL

ライフタイム注釈と省略規則を図でつかむ

入力・所有する側

参照の有効期間	owner
dangling防止	owner

処理・借りる側

省略規則	borrow
入力と出力の関係	borrow

ライフタイムは参照が有効な期間の関係を表し、値を長生きさせる記法ではありません。多くは省略規則で推論され、複数参照から一つを返すときに関係を注釈します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	参照の有効期間	参照の有効期間 — 入力と前提を確認し、境界を曖昧にしない。
2	dangling防止	dangling防止 — 値がどの順で変換されるかを追跡する。
3	省略規則	省略規則 — 失敗を再現し、型またはログから原因を特定する。
4	入力と出力の関係	入力と出力の関係 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

ライフタイム注釈と省略規則とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05 · MINIMUM CODE

まず動く最小コード

ライフタイム注釈と省略規則だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn longest<'a>(left: &'a str, right: &'a str) -> &'a str {
    if left.len() >= right.len() { left } else { right }
}
fn main() {
    let a = String::from("database");
    let b = String::from("api");
    println!("{}", longest(&a, &b));
}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn longest<'a>(left: &'a str, right: &'a str) -> &'a str {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>if left.len() >= right.len() { left } else { right }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let a = String::from("database");</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力参照の有効期間の条件を満たすか確認する。
- ✓ dangling防止を実行し、途中の型と状態を追う。
- ✓ 省略規則で失敗を成功経路から分離する。
- ✓ 入力と出力の関係により、出力と副作用を検証する。

型を声に出す ライフタイム注釈と省略規則に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

database

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn strip_prefix<'a>(value: &'a str, prefix: &str) -> Option<&'a str> {
    value.strip_prefix(prefix)
}
fn main() { assert_eq!(strip_prefix("Bearer token", "Bearer "), Some("token")); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

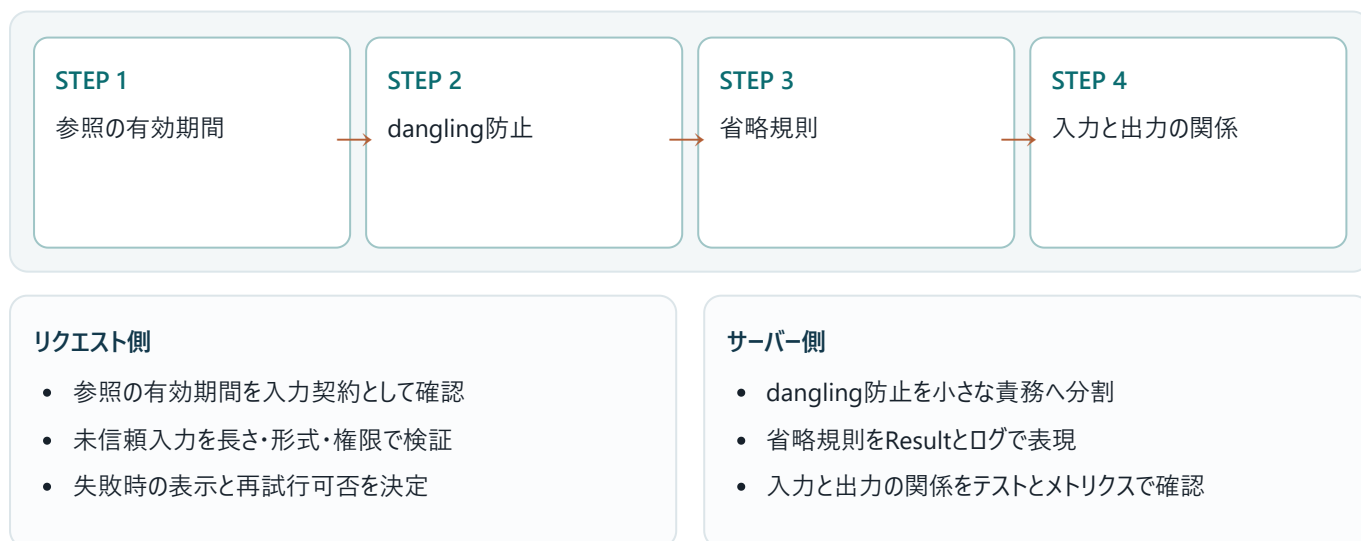
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05 · BACKEND APPLICATION

バックエンドのどこで使うか

リクエストから借りた&strをアプリ全体状態へ保存できない理由を理解し、必要なら所有するStringへ変換します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	参照の有効期間に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	dangling防止の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	省略規則または入力と出力の関係の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error[E0106]: missing lifetime specifier
help: consider introducing a named lifetime parameter
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

ライフタイム注釈と省略規則を再現して切り分ける

ライフタイム注釈と省略規則の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 参照の有効期間
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: dangling防止
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 省略規則
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 入力と出力の関係

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	参照の有効期間	参照の有効期間 — 入力と前提を確認し、境界を曖昧にしない。
2	dangling防止	dangling防止 — 値がどの順で変換されるかを追跡する。
3	省略規則	省略規則 — 失敗を再現し、型またはログから原因を特定する。
4	入力と出力の関係	入力と出力の関係 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 ライフタイム注釈は参照同士の有効期間の関係を説明します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

prefixとvalueを受け、valueがprefixで始まるときだけvalue由来の&strを返す関数へライフタイムを付けてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn strip_prefix<'a>(value: &'a str, prefix: &str) -> Option<&'a str> {  
    value.strip_prefix(prefix)  
}  
fn main() { assert_eq!(strip_prefix("Bearer token", "Bearer "), Some("token")); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ ライフタイム注釈は参照同士の有効期間の関係を説明します。
- ✓ ライフタイム注釈と省略規則とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

参照を持つstruct・static・所有へ切替

UNIT 06 ・ CONCEPT

参照を持つstruct・static・所有へ切替

参照フィールドを持つstructには、その参照がstructより長く有効である関係が必要です。'staticはプログラム全体に有効な参照であり、何にでも付ける解決策ではありません。

この単元の到達点

- structのライフタイム
- 借用データの束
- 'staticの意味
- String所有への切替

なぜバックエンドで必要か

リクエスト処理中だけ使う解析結果は借用で効率化できますが、async taskやAppStateへ持ち越す値は所有するStringにします。

入力・所有する側

structのライフタイム	owner
借用データの束	owner

処理・借りる側

'staticの意味	borrow
String所有への切替	borrow

先に答え

短い処理内は借用、処理境界を越えて保存するなら所有する型へ変えます。

確認する問い

参照を持つstruct・static・所有へ切替を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 06 ・ MENTAL MODEL

参照を持つstruct・static・所有へ切替を図でつかむ

入力・所有する側

structのライフタイム	owner
借用データの束	owner

処理・借りる側

'staticの意味	borrow
String所有への切替	borrow

参照フィールドを持つstructには、その参照がstructより長く有効である関係が必要です。'staticはプログラム全体に有効な参照であり、何にでも付ける解決策ではありません。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	structのライフタイム	structのライフタイム — 入力と前提を確認し、境界を曖昧にしない。
2	借用データの束	借用データの束 — 値がどの順で変換されるかを追跡する。
3	'staticの意味	'staticの意味 — 失敗を再現し、型またはログから原因を特定する。
4	String所有への切替	String所有への切替 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

参照を持つstruct・static・所有へ切替とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06 · MINIMUM CODE

まず動く最小コード

参照を持つstruct・static・所有へ切替だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(Debug)]
struct Header<'a> { name: &'a str, value: &'a str }
fn parse(raw: &str) -> Option<Header<'_>> {
    let (name, value) = raw.split_once(':')?;
    Some(Header { name: name.trim(), value: value.trim() })
}
fn main() { println!("{:?}", parse("Content-Type: text/html")); }
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Debug)]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>struct Header<'a> { name: &'a str, value: &'a str }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn parse(raw: &str) -> Option<Header<'_>> {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let (name, value) = raw.split_once(':');</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>Some(Header { name: name.trim(), value: value.trim() })</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力があるstructのライフタイムの条件を満たすか確認する。
- ✓ 借用データの束を実行し、途中の型と状態を追う。
- ✓ 'staticの意味で失敗を成功経路から分離する。
- ✓ String所有への切替により、出力と副作用を検証する。

型を声に出す 参照を持つstruct・static・所有へ切替に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
Some(Header { name: "Content-Type", value: "text/html" })
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
struct HeaderView<'a> { name: &'a str, value: &'a str }
#[derive(Debug)] struct OwnedHeader { name: String, value: String }
impl From<HeaderView<'_>> for OwnedHeader {
    fn from(h: HeaderView<'_>) -> Self { Self { name: h.name.into(), value: h.value.into() } }
}
fn main() { let owned = OwnedHeader::from(HeaderView { name: "x-id", value: "7" }); println!("{owned:?}"); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06 · BACKEND APPLICATION

バックエンドのどこで使うか

リクエスト処理中だけ使う解析結果は借用で効率化できますが、async taskやAppStateへ持ち越す値は所有するStringにします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	structのライフタイムに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	借用データの束の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	'staticの意味またはString所有への切替の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

参照を持つstruct・static・所有へ切替を再現して切り分ける

参照を持つstruct・static・所有へ切替の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: structのライフタイム
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 借用データの束
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 'staticの意味
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: String所有への切替

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	structのライフタイム	structのライフタイム — 入力と前提を確認し、境界を曖昧にしない。
2	借用データの束	借用データの束 — 値がどの順で変換されるかを追跡する。
3	'staticの意味	'staticの意味 — 失敗を再現し、型またはログから原因を特定する。
4	String所有への切替	String所有への切替 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 短い処理内は借用、処理境界を越えて保存するなら所有する型へ変えます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

借用するHeaderViewと、保存可能なOwnedHeaderを相互変換してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
struct HeaderView<'a> { name: &'a str, value: &'a str }
#[derive(Debug)] struct OwnedHeader { name: String, value: String }
impl From<HeaderView<'_>> for OwnedHeader {
    fn from(h: HeaderView<'_>) -> Self { Self { name: h.name.into(), value: h.value.into() } }
}
fn main() { let owned = OwnedHeader::from(HeaderView { name: "x-id", value: "7" }); println!("{owned:?}"); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 短い処理内は借用、処理境界を越えて保存するなら所有する型へ変えます。
- ✓ 参照を持つstruct・static・所有へ切替とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

NLL・再借用・借用範囲を短くする

UNIT 07・CONCEPT

NLL・再借用・借用範囲を短くする

Non-Lexical Lifetimesにより借用はブロック末尾ではなく最後の使用箇所ですべて終了できます。それでも長い`&mut`借用は他の操作を妨げるため、検索と更新を短い式へ閉じ込めます。

この単元の到達点

- 最後の使用で借用終了
- 再借用
- `&mut`の短縮
- 処理の分割

なぜバックエンドで必要か

AppStateやキャッシュのロックガードを`await`より前に`drop`し、同期ロックを非同期待機中まで保持しない設計へつながります。

入力・所有する側

最後の使用で借用終了	owner
再借用	owner

処理・借りる側

<code>&mut</code> の短縮	borrow
処理の分割	borrow

先に答え

借用は最後の使用で終わりますが、更新処理を小さくして範囲を明確にします。

確認する問い

NLL・再借用・借用範囲を短くするを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 07・MENTAL MODEL

NLL・再借用・借用範囲を短くするを図でつかむ

入力・所有する側

最後の使用で借用終了	owner
再借用	owner

処理・借りる側

<code>&mut</code> の短縮	borrow
処理の分割	borrow

Non-Lexical Lifetimesにより借用はブロック末尾ではなく最後の使用箇所ですべて終了できます。それでも長い`&mut`借用は他の操作を妨げるため、検索と更新を短い式へ閉じ込めます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	最後の使用で借用終了	最後の使用で借用終了 — 入力と前提を確認し、境界を曖昧にしない。
2	再借用	再借用 — 値がどの順で変換されるかを追跡する。
3	&mutの短縮	&mutの短縮 — 失敗を再現し、型またはログから原因を特定する。
4	処理の分割	処理の分割 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

NLL・再借用・借用範囲を短くするとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07 · MINIMUM CODE

まず動く最小コード

NLL・再借用・借用範囲を短くするだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn main() {
    let mut values = vec![1, 2, 3];
    let first = &values[0];
    println!("first={first}"); // firstの最後の使用
    values.push(4);           // ここでは共有借用が終了済み
    println!("{values:?}");
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let mut values = vec![1, 2, 3];</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let first = &values[0];</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>println!("first={first}"); // firstの最後の使用</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>values.push(4); // ここでは共有借用が終了済み</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力最後の使用で借用終了の条件を満たすか確認する。
- ✓ 再借用を実行し、途中の型と状態を追う。
- ✓ &mutの短縮で失敗を成功経路から分離する。
- ✓ 処理の分割により、出力と副作用を検証する。

型を声に出す NLL・再借用・借用範囲を短くするに關係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
first=1
[1, 2, 3, 4]
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use std::collections::HashMap;
fn main() {
    let mut map = HashMap::from([("a", 1), ("b", 2)]);
    if let Some(value) = map.get_mut("a") { *value += 1; }
    let total: i32 = map.values().sum();
    assert_eq!(total, 4);
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

AppStateやキャッシュのロックガードをawaitより前にdropし、同期ロックを非同期待機中まで保持しない設計へつながります。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	最後の使用で借用終了に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	再借用の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	&mutの短縮または処理の分割の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

NLL・再借用・借用範囲を短くするを再現して切り分ける

NLL・再借用・借用範囲を短くするの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 最後の使用で借用終了
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 再借用
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: &mutの短縮
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 処理の分割

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	最後の使用で借用終了	最後の使用で借用終了 — 入力と前提を確認し、境界を曖昧にしない。
2	再借用	再借用 — 値がどの順で変換されるかを追跡する。
3	&mutの短縮	&mutの短縮 — 失敗を再現し、型またはログから原因を特定する。
4	処理の分割	処理の分割 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 借用は最後の使用で終わりますが、更新処理を小さくして範囲を明確にします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

HashMapの一件を更新した後、同じMap全体を読み取るコードを借用範囲が短くなるよう書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use std::collections::HashMap;
fn main() {
    let mut map = HashMap::from([("a", 1), ("b", 2)]);
    if let Some(value) = map.get_mut("a") { *value += 1; }
    let total: i32 = map.values().sum();
    assert_eq!(total, 4);
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 借用は最後の使用で終わりますが、更新処理を小さくして範囲を明確にします。
- ✓ NLL・再借用・借用範囲を短くするとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

Box<T>・再帰型・trait object

UNIT 08・CONCEPT

Box<T>・再帰型・trait object

Box<T>は値をヒープへ置き、ポインタ自体は固定サイズになります。再帰型のサイズを有限にし、大きな値の所有権移動を軽くし、dyn Traitの実体を保持できます。

この単元の到達点

- ヒープ所有
- 再帰型の間接化
- 固定サイズ
- Box<dyn Trait>

なぜバックエンドで必要か

複雑な検索条件やポリシー木を再帰enumで表せます。動的ディスパッチが必要な境界ではBox<dyn Trait>を使います。

入力・所有する側

ヒープ所有	owner
再帰型の間接化	owner

処理・借りる側

固定サイズ	borrow
Box<dyn Trait>	borrow

先に答え

Boxは所有したまま間接化し、再帰型と動的traitのサイズ問題を解きます。

確認する問い

Box<T>・再帰型・trait objectを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 08・MENTAL MODEL

Box<T>・再帰型・trait objectを図でつかむ

入力・所有する側

ヒープ所有	owner
再帰型の間接化	owner

処理・借りる側

固定サイズ	borrow
Box<dyn Trait>	borrow

Box<T>は値をヒープへ置き、ポインタ自体は固定サイズになります。再帰型のサイズを有限にし、大きな値の所有権移動を軽くし、dyn Traitの実体を保持できます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	ヒープ所有	ヒープ所有 — 入力と前提を確認し、境界を曖昧にしない。
2	再帰型の間接化	再帰型の間接化 — 値がどの順で変換されるかを追跡する。
3	固定サイズ	固定サイズ — 失敗を再現し、型またはログから原因を特定する。
4	Box<dyn Trait>	Box<dyn Trait> — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Box<T>・再帰型・trait objectとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08・MINIMUM CODE

まず動く最小コード

Box<T>・再帰型・trait objectだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(Debug)]
enum Filter {
    Done(bool),
    And(Box<Filter>, Box<Filter>),
}
fn main() {
    let filter = Filter::And(Box::new(Filter::Done(true)), Box::new(Filter::Done(false)));
    println!("{}", filter);
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Debug)]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>enum Filter {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>Done(bool),</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>And(Box<Filter>, Box<Filter>),</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力ヒープ所有の条件を満たすか確認する。
- ✓ 再帰型の間接化を実行し、途中の型と状態を追う。
- ✓ 固定サイズで失敗を成功経路から分離する。
- ✓ `Box<dyn Trait>`により、出力と副作用を検証する。

型を声に出す `Box<T>`・再帰型・trait objectに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

And(Done(true), Done(false))

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
trait Logger { fn log(&self, message: &str); }
struct StdoutLogger;
impl Logger for StdoutLogger { fn log(&self, message: &str) { println!("{message}"); } }
fn run(logger: Box<dyn Logger>) { logger.log("server started"); }
fn main() { run(Box::new(StdoutLogger)); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08・BACKEND APPLICATION

バックエンドのどこで使うか

複雑な検索条件やポリシー木を再帰enumで表せます。動的ディスパッチが必要な境界ではBox<dyn Trait>を使います。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	ヒープ所有に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	再帰型の間接化の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	固定サイズまたはBox<dyn Trait>の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

Box<T>・再帰型・trait objectを再現して切り分ける

Box<T>・再帰型・trait objectの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: ヒープ所有
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 再帰型の間接化
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 固定サイズ
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: Box<dyn Trait>

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	ヒープ所有	ヒープ所有 — 入力と前提を確認し、境界を曖昧にしない。
2	再帰型の間接化	再帰型の間接化 — 値がどの順で変換されるかを追跡する。
3	固定サイズ	固定サイズ — 失敗を再現し、型またはログから原因を特定する。
4	Box<dyn Trait>	Box<dyn Trait> — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 Boxは所有したまま間接化し、再帰型と動的traitのサイズ問題を解きます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

Logger traitを定義し、StdoutLoggerをBox<dyn Logger>として関数へ渡してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
trait Logger { fn log(&self, message: &str); }
struct StdoutLogger;
impl Logger for StdoutLogger { fn log(&self, message: &str) { println!("{message}"); } }
fn run(logger: Box<dyn Logger>) { logger.log("server started"); }
fn main() { run(Box::new(StdoutLogger)); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ Boxは所有したまま間接化し、再帰型と動的traitのサイズ問題を解きます。
- ✓ Box<T>・再帰型・trait objectとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

Rc<T>・RefCell<T>・内部可変性

UNIT 09・CONCEPT

Rc<T>・RefCell<T>・内部可変性

Rc<T>は単一スレッドで所有者を複数にし、RefCell<T>は借用規則を実行時に検査します。コンパイル時保証を弱めるため、UI木やテスト用Fakeなど限定した場所で使います。

この単元の到達点

- 参照カウント
- 単一スレッド
- 実行時借用検査
- borrow panic

なぜバックエンドで必要か

非同期マルチスレッドサーバーの共有状態にはRc/RefCellを使えません。Send/Syncが必要なAppStateはArcと適切な同期原語を使います。

入力・所有する側

参照カウント	owner
単一スレッド	owner

処理・借りる側

実行時借用検査	borrow
borrow panic	borrow

先に答え

RefCellは借用規則を消すのではなく実行時へ移すため、ガード範囲を短くします。

確認する問い

Rc<T>・RefCell<T>・内部可変性を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 09・MENTAL MODEL

Rc<T>・RefCell<T>・内部可変性を図でつかむ

入力・所有する側

参照カウント	owner
単一スレッド	owner

処理・借りる側

実行時借用検査	borrow
borrow panic	borrow

Rc<T>は単一スレッドで所有者を複数にし、RefCell<T>は借用規則を実行時に検査します。コンパイル時保証を弱めるため、UI木やテスト用Fakeなど限定した場所で使います。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	参照カウント	参照カウント — 入力と前提を確認し、境界を曖昧にしない。
2	単一スレッド	単一スレッド — 値がどの順で変換されるかを追跡する。
3	実行時借用検査	実行時借用検査 — 失敗を再現し、型またはログから原因を特定する。
4	borrow panic	borrow panic — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

Rc<T>・RefCell<T>・内部可変性とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09・MINIMUM CODE まず動く最小コード

Rc<T>・RefCell<T>・内部可変性だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use std::{cell::RefCell, rc::Rc};
fn main() {
    let events = Rc::new(RefCell::new(Vec::<String>::new()));
    let writer = Rc::clone(&events);
    writer.borrow_mut().push("created".into());
    println!("{:?}", events.borrow());
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::{cell::RefCell, rc::Rc};</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let events = Rc::new(RefCell::new(Vec:: <String>::new()));</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let writer = Rc::clone(&events);</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>writer.borrow_mut().push("created".into());</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が参照カウントの条件を満たすか確認する。
- ✓ 単一スレッドを実行し、途中の型と状態を追う。
- ✓ 実行時借用検査で失敗を成功経路から分離する。
- ✓ borrow panicにより、出力と副作用を検証する。

型を声に出す `Rc<T>`・`RefCell<T>`・内部可変性に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

`["created"]`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use std::cell::RefCell;
fn main() {
    let values = RefCell::new(vec![1]);
    { values.borrow_mut().push(2); }
    { values.borrow_mut().push(3); }
    assert_eq!(*values.borrow(), vec![1, 2, 3]);
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

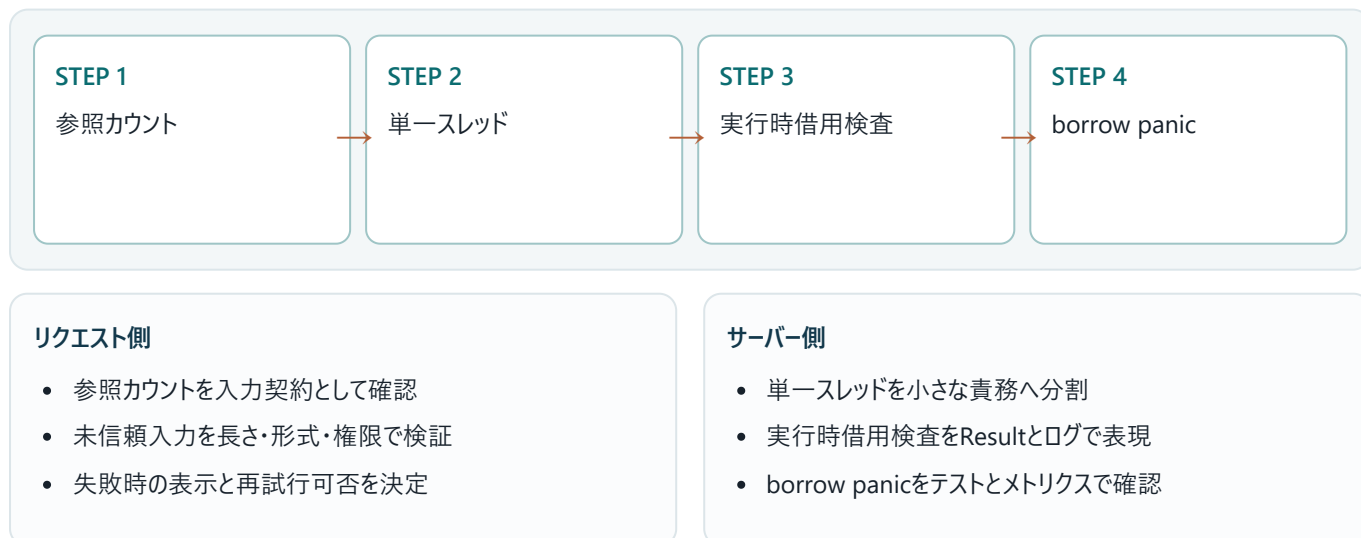
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09 · BACKEND APPLICATION

バックエンドのどこで使うか

非同期マルチスレッドサーバーの共有状態にはRc/RefCellを使えません。Send/Syncが必要なAppStateはArcと適切な同期原語を使います。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	参照カウントに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	単一スレッドの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	実行時借用検査またはborrow panicの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
thread 'main' panicked at 'already borrowed: BorrowMutError'
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

Rc<T>・RefCell<T>・内部可変性を再現して切り分ける

Rc<T>・RefCell<T>・内部可変性の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 参照カウント
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 単一スレッド
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 実行時借用検査
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: borrow panic

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	参照カウント	参照カウント — 入力と前提を確認し、境界を曖昧にしない。
2	単一スレッド	単一スレッド — 値がどの順で変換されるかを追跡する。
3	実行時借用検査	実行時借用検査 — 失敗を再現し、型またはログから原因を特定する。
4	borrow panic	borrow panic — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 RefCellは借用規則を消すのではなく実行時へ移すため、ガード範囲を短くします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

RefCellの可変借用を二重にしてpanicを再現した後、借用ガードのスコープを分けて修正してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use std::cell::RefCell;
fn main() {
    let values = RefCell::new(vec![1]);
    { values.borrow_mut().push(2); }
    { values.borrow_mut().push(3); }
    assert_eq!(*values.borrow(), vec![1, 2, 3]);
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ RefCellは借用規則を消すのではなく実行時へ移すため、ガード範囲を短くします。
- ✓ Rc<T>・RefCell<T>・内部可変性とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

Arc<T>・Mutex<T>・Send・Sync

UNIT 10・CONCEPT

Arc<T>・Mutex<T>・Send・Sync

Arc<T>はスレッド安全な参照カウント、Mutex<T>は共有可変状態への排他アクセスを提供します。Sendは所有権を別スレッドへ移せること、Syncは&Tを複数スレッドで共有できることを表します。

この単元の到達点

- Arcの共有所有
- MutexGuard
- Send境界
- Sync境界

なぜバックエンドで必要か

TopcoatのAppStateにDB handleや設定をArcで共有します。Mutexをawait中に保持せず、DBのトランザクションやchannelを優先できないか検討します。

入力・所有する側

Arcの共有所有	owner
MutexGuard	owner

処理・借りる側

Send境界	borrow
Sync境界	borrow

先に答え

マルチスレッド共有はArc、可変状態は最小範囲の同期で守り、Send/Sync境界を確認します。

確認する問い

Arc<T>・Mutex<T>・Send・Syncを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 10・MENTAL MODEL

Arc<T>・Mutex<T>・Send・Syncを図でつかむ

入力・所有する側

Arcの共有所有	owner
MutexGuard	owner

処理・借りる側

Send境界	borrow
Sync境界	borrow

Arc<T>はスレッド安全な参照カウント、Mutex<T>は共有可変状態への排他アクセスを提供します。Sendは所有権を別スレッドへ移せること、Syncは&Tを複数スレッドで共有できることを表します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Arcの共有所有	Arcの共有所有 — 入力と前提を確認し、境界を曖昧にしない。
2	MutexGuard	MutexGuard — 値がどの順で変換されるかを追跡する。
3	Send境界	Send境界 — 失敗を再現し、型またはログから原因を特定する。
4	Sync境界	Sync境界 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Arc<T>・Mutex<T>・Send・Syncとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

Arc<T>・Mutex<T>・Send・Syncだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use std::{sync::{Arc, Mutex}, thread};
fn main() {
    let count = Arc::new(Mutex::new(0_u64));
    let handles: Vec<_> = (0..4).map(|_| {
        let count = Arc::clone(&count);
        thread::spawn(move || *count.lock().unwrap() += 1)
    }).collect();
    for h in handles { h.join().unwrap(); }
    println!("count={}", *count.lock().unwrap());
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::{sync::{Arc, Mutex}, thread};</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let count = Arc::new(Mutex::new(0_u64));</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let handles: Vec<_> = (0..4).map(_ {</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let count = Arc::clone(&count);</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力 Arc の共有所有の条件を満たすか確認する。
- ✓ MutexGuard を実行し、途中の型と状態を追う。
- ✓ Send 境界で失敗を成功経路から分離する。
- ✓ Sync 境界により、出力と副作用を検証する。

型を声に出す Arc<T>・Mutex<T>・Send・Sync に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

count=4

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 10・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use std::{sync::{Arc, Mutex}, thread};
fn main() {
    let events = Arc::new(Mutex::new(Vec::new()));
    let handles: Vec<_> = (0..4).map(|id| {
        let events = Arc::clone(&events);
        thread::spawn(move || events.lock().unwrap().push(format!("event-{{id}}")))
    }).collect();
    for h in handles { h.join().unwrap(); }
    assert_eq!(events.lock().unwrap().len(), 4);
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10・BACKEND APPLICATION

バックエンドのどこで使うか

TopcoatのAppStateにDB handleや設定をArcで共有します。Mutexをawait中に保持せず、DBのトランザクションやchannelを優先できないか検討します。



リクエスト側

- Arcの共有所有を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- MutexGuardを小さな責務へ分割
- Send境界をResultとログで表現
- Sync境界をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Arcの共有所有に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	MutexGuardの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	Send境界またはSync境界の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

Arc<T>・Mutex<T>・Send・Syncを再現して切り分ける

Arc<T>・Mutex<T>・Send・Syncの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Arcの共有所有
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: MutexGuard
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Send境界
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: Sync境界

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Arcの共有所有	Arcの共有所有 — 入力と前提を確認し、境界を曖昧にしない。
2	MutexGuard	MutexGuard — 値がどの順で変換されるかを追跡する。
3	Send境界	Send境界 — 失敗を再現し、型またはログから原因を特定する。
4	Sync境界	Sync境界 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 マルチスレッド共有はArc、可変状態は最小範囲の同期で守り、Send/Sync境界を確認します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

Arc<Mutex<Vec<String>>>へ4スレッドからイベントを書き、件数と内容を検証してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use std::{sync::{Arc, Mutex}, thread};
fn main() {
    let events = Arc::new(Mutex::new(Vec::new()));
    let handles: Vec<_> = (0..4).map(|id| {
        let events = Arc::clone(&events);
        thread::spawn(move || events.lock().unwrap().push(format!("event-{{id}}")))
    }).collect();
    for h in handles { h.join().unwrap(); }
    assert_eq!(events.lock().unwrap().len(), 4);
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ マルチスレッド共有はArc、可変状態は最小範囲の同期で守り、Send/Sync境界を確認します。
- ✓ Arc<T>・Mutex<T>・Send・Syncとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

所有権を意識したスレッド安全なインメモリ NoteRepositoryを作る

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	スタック・ヒープ・スコープ・Drop	所有者のスコープ終了で資源を一度だけ解放する仕組みがRAIIです。
02	Move・Copy・Clone	所有権を移すのは責務移譲、借りるのは一時利用、cloneは費用を払う複製です。
03	共有参照& Tと可変参照&mut T	読む借用は共有でき、書く借用は排他的です。借用範囲を短く保ちます。
04	スライスと借用APIの設計	所有を要求しない関数は&strや&[T]を受け、呼び出し側の選択肢を広げます。
05	ライフタイム注釈と省略規則	ライフタイム注釈は参照同士の有効期間の関係を説明します。
06	参照を持つstruct・static・所有へ切替	短い処理内は借用、処理境界を越えて保存するなら所有する型へ変えます。
07	NLL・再借用・借用範囲を短くする	借用は最後の使用で終わりますが、更新処理を小さくして範囲を明確にします。
08	Box<T>・再帰型・trait object	Boxは所有したまま間接化し、再帰型と動的traitのサイズ問題を解きます。
09	Rc<T>・RefCell<T>・内部可変性	RefCellは借用規則を消すのではなく実行時へ移すため、ガード範囲を短くします。
10	Arc<T>・Mutex<T>・Send・Sync	マルチスレッド共有はArc、可変状態は最小範囲の同期で守り、Send/Sync境界を確認します。

完了条件

- ✓ cloneを必要箇所だけに限定した
- ✓ 公開APIは借用と所有を意図的に選んだ
- ✓ ロックを保持する範囲が短い
- ✓ 並行テストでデータ競合なく動作した

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. スタック・ヒープ・スコープ・Dropを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. Move・Copy・Cloneを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. 共有参照& Tと可変参照&mut Tを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. スライスと借用APIの設計を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. ライフタイム注釈と省略規則を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. 参照を持つstruct・static・所有へ切替を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. NLL・再借用・借用範囲を短くするを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. Box<T>・再帰型・trait objectを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. Rc<T>・RefCell<T>・内部可変性を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. Arc<T>・Mutex<T>・Send・Syncを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. スタック・ヒープ・スコープ・Drop

スタック・ヒープ・スコープ・Dropとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 所有者のスコープ終了で資源を一度だけ解放する仕組みがRAIIです。

02. Move・Copy・Clone

Move・Copy・Cloneとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 所有権を移すのは責務移譲、借りるのは一時利用、cloneは費用を払う複製です。

03. 共有参照& Tと可変参照&mut T

共有参照& Tと可変参照&mut Tとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 読む借用は共有でき、書く借用は排他的です。借用範囲を短く保ちます。

04. スライスと借用APIの設計

スライスと借用APIの設計とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 所有を要求しない関数は&strや&[T]を受け、呼び出し側の選択枝を上げます。

05. ライフタイム注釈と省略規則

ライフタイム注釈と省略規則とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: ライフタイム注釈は参照同士の有効期間の関係を説明します。

06. 参照を持つstruct・static・所有へ切替

参照を持つstruct・static・所有へ切替とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 短い処理内は借用、処理境界を越えて保存するなら所有する型へ変えます。

07. NLL・再借用・借用範囲を短くする

NLL・再借用・借用範囲を短くするとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 借用は最後の使用で終わりますが、更新処理を小さくして範囲を明確にします。

08. Box<T>・再帰型・trait object

Box<T>・再帰型・trait objectとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: Boxは所有したまま間接化し、再帰型と動的traitのサイズ問題を解きます。

09. Rc<T>・RefCell<T>・内部可変性

Rc<T>・RefCell<T>・内部可変性とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: RefCellは借用規則を消すのではなく実行時へ移すため、ガード範囲を短くします。

10. Arc<T>・Mutex<T>・Send・Sync

Arc<T>・Mutex<T>・Send・Syncとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: マルチスレッド共有はArc、可変状態は最小範囲の同期で守り、Send/Sync境界を確認します。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
スタック・ヒープ・スコープ・Drop	所有者のスコープ終了で資源を一度だけ解放する仕組みがRAIIです。	DB接続、ファイル、ロックはRAIIでスコープ終了時に解放されます。解放処理を手動の全return経路へ書く必要がありません。
Move・Copy・Clone	所有権を移すのは責務移譲、借りるのは一時利用、cloneは費用を払う複製です。	DTOをユースケースへMoveすると、その後の誤使用を防げます。大きなVecを無意識にcloneするとリクエストごとのCPU・メモリ負荷になります。
共有参照& Tと可変参照&mut T	読む借用は共有でき、書く借用は排他的です。借用範囲を短く保ちます。	複数ハンドラが共有する状態は不変参照で読み、更新はDBトランザクションやMutexなど排他境界へ集めます。
スライスと借用APIの設計	所有を要求しない関数は&strや&[T]を受け、呼び出し側の選択肢を広げます。	フォームのStringを検証関数へ&strで渡し、DB行のVecを集計関数へ&[T]で渡すと、不要な所有権移動とcloneを避けられます。
ライフタイム注釈と省略規則	ライフタイム注釈は参照同士の有効期間の関係を説明します。	リクエストから借りた&strをアプリ全体状態へ保存できない理由を理解し、必要なら所有するStringへ変換します。
参照を持つstruct・static・所有へ切替	短い処理内は借用、処理境界を越えて保存するなら所有する型へ変えます。	リクエスト処理中だけ使う解析結果は借用で効率化できますが、async taskやAppStateへ持ち越す値は所有するStringにします。
NLL・再借用・借用範囲を短くする	借用は最後の使用で終わりますが、更新処理を小さくして範囲を明確にします。	AppStateやキャッシュのロックガードをawaitより前にdropし、同期ロックを非同期待機中まで保持しない設計へつな갑니다。
Box<T>・再帰型・trait object	Boxは所有したまま間接化し、再帰型と動的traitのサイズ問題を解きます。	複雑な検索条件やポリシー木を再帰enumで表せます。動的デイスパッチが必要な境界ではBox<dyn Trait>を使います。
Rc<T>・RefCell<T>・内部可変性	RefCellは借用規則を消すのではなく実行時へ移すため、ガード範囲を短くします。	非同期マルチスレッドサーバーの共有状態にはRc/RefCellを使えません。Send/Syncが必要なAppStateはArcと適切な同期原語を使います。
Arc<T>・Mutex<T>・Send・Sync	マルチスレッド共有はArc、可変状態は最小範囲の同期で守り、Send/Sync境界を確認します。	TopcoatのAppStateにDB handleや設定をArcで共有します。Mutexをawait中に保持せず、DBのトランザクションやchannelを優先できないか検討します。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。